

KUBERNETES / K3S — HANDS-ON

Deployments, Services, Config & Secrets

Five worked examples, each building on the last — from a bare nginx pod to a configurable app with externalised config and registry credentials.

Day 01 · run on k3s v1.36.4+k3s1 · examples ex01 → ex05

Where we're going — one new idea per step

ex01

nginx

Deployment + Service — the smallest workload that serves traffic

ex02

+ Redis

Two Deployments talking over a Service; discovery by DNS name

ex03

+ env var

Feed the app its config through an environment variable (REDIS_HOST)

ex04

+ ConfigMap

Move that config out of the pod spec into a reusable object

ex05

+ Secret

Docker Hub credentials as a Secret, used as imagePullSecrets

Same app image throughout: [vishwacloudlab/pythonapp:v4-var](#) (Flask, port 8000)

What you already have running

- Single-node k3s on Ubuntu 24.04
- Control-plane + workloads on one VM
- kubectl talks to it via ~/.kube/config
- Bundled: containerd, Flannel (CNI), CoreDNS, Traefik, ServiceLB, local-path storage

```
$ kubectl get nodes
NAME          STATUS    ROLES          AGE
lab-g2-vm2    Ready    control-plane,master  3h

$ kubectl get pods -n kube-system
coredns-...    1/1    Running
traefik-...    1/1    Running
metrics-server-...  1/1    Running
```

All examples run in their own namespace:

```
kubectl create ns day01
kubectl config set-context --current --namespace=day01
```

Four objects, 90 seconds

Pod

One or more containers that share a network + lifecycle. The unit that runs. Ephemeral — never create directly.

ReplicaSet

Keeps exactly N identical pods alive. You rarely touch it — the Deployment owns it.

Deployment

Declares desired state (image, replicas, env). Rolls out changes and rolls back. This is what you write.

Service

Stable virtual IP + DNS name. Load-balances to the pods that match its label selector, even as pods come and go.

Deployment → manages → ReplicaSet → creates → Pods ← selected by → Service

The smallest thing that serves traffic

nginx.yaml (Deployment)

```
kind: Deployment
spec:
  replicas: 2
  selector:
    matchLabels: { app: nginx }
  template:
    metadata:
      labels: { app: nginx }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports: [{ containerPort: 80 }]
```

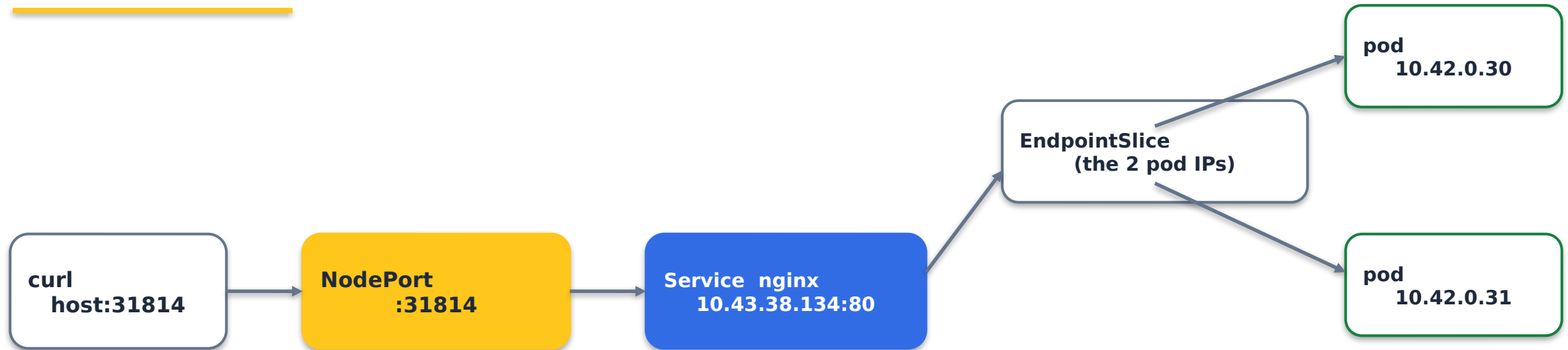
nginx.yaml (Service)

```
kind: Service
spec:
  type: NodePort
  selector: { app: nginx }
  ports:
    - port: 80
      targetPort: 80
```

- selector must match the pod labels — that is the only link
- replicas: 2 → ReplicaSet keeps two pods
- NodePort opens one port (30000–32767)

```
$ kubectl apply -f nginx.yaml
```

How a request reaches a pod



- kube-proxy programmes the NodePort → Service → pod hops as iptables rules.
- The Service tracks healthy pods through its selector; the EndpointSlice is that live list.

```
kubectl get endpointslices • curl -s -o /dev/null -w '%{http_code}' host:31814 → 200
```

Verified run

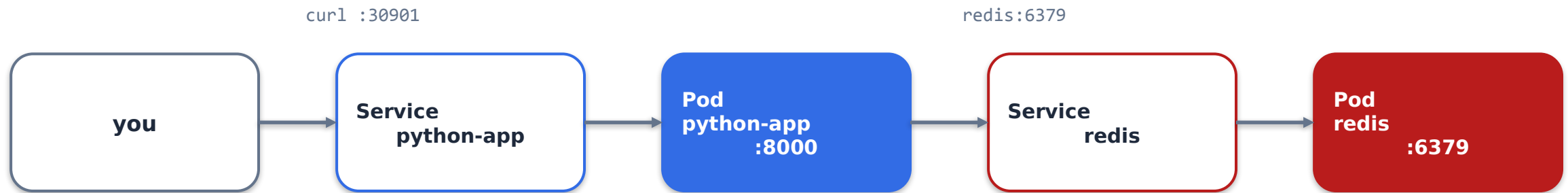
```
$ kubectl get deploy,pod,svc -l app=nginx
deployment.apps/nginx 2/2 2 up-to-date 2 available
pod/nginx-795669cc7d-nvmvf 1/1 Running
pod/nginx-795669cc7d-qr9bw 1/1 Running
service/nginx NodePort 10.43.38.134 80:31814/TCP

$ kubectl get endpoints nginx
nginx 10.42.0.30:80,10.42.0.31:80 # one entry per pod

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://192.168.230.103:31814
+HTTP 200
```

Delete one pod → the ReplicaSet recreates it within seconds. That's the control loop.

Add a backend the app depends on



app.py

```
redis_host = os.getenv("REDIS_HOST", "redis")
redis = Redis(host=redis_host, port=6379)
@app.route('/'): count = redis.incr('hits') → "Hello World! I have been seen N times."
```

Apply redis.yaml, then python-app.yaml. No env set yet — the next slide shows why it still works.

Service discovery = DNS

Every Service gets an in-cluster DNS name:

`<service>.<namespace>.svc.cluster.local` (short form: just `<service>` in the same namespace)

```
$ kubectl exec deploy/python-app -- \
    getent hosts redis
10.43.98.115    redis.day01.svc.cluster.local

$ kubectl exec deploy/python-app -- \
    sh -c 'echo REDIS_HOST=${REDIS_HOST:-<unset>}'
REDIS_HOST=<unset>
```

Works by convention

image default `REDIS_HOST = "redis"`
Redis Service is named `redis`
→ they line up, zero config needed

Fragile: it only works while the name happens to match. ex03 makes the dependency explicit.

Tell the app where Redis is

ex03 · redis.yaml + python-app.yaml

```
kind: Service
metadata:
  name: redis-primary      # renamed!
---
kind: Deployment
spec:
  template:
    spec:
      containers:
        - name: python-app
          env:
            - name: REDIS_HOST
              value: "redis-primary"
```

- The Service is now redis-primary.
- The image default "redis" no longer resolves.
- So we must pass REDIS_HOST explicitly.
- env: is the most direct way — one key, one value, right in the pod spec.

```
kubectl exec deploy/python-app -- \
  sh -c 'echo $REDIS_HOST'
+redis-primary
```

Prove the dependency — remove it, watch it break

```
$ kubectl set env deploy/python-app REDIS_HOST-      # unset it
$ kubectl rollout status deploy/python-app

$ curl http://<pod-ip>:8000/
HTTP/1.1 500 INTERNAL SERVER ERROR

$ kubectl logs deploy/python-app
File "/code/app.py", line 12, in hello
    count = redis.incr('hits')
redis.exceptions.ConnectionError:
    Error -2 connecting to redis:6379. Name does not resolve.
```

Put REDIS_HOST=redis-primary back → rollout → 200 again. Config is now a first-class, testable part of the deployment.

Lift config out of the pod spec

configmap.yaml

```
kind: ConfigMap
metadata:
  name: python-app-config
data:
  REDIS_HOST: redis-primary
  LOG_LEVEL: info
```

```
# C · mount as files
volumes:
  - name: cfg
    configMap: { name: python-app-config }
# -> file /etc/app/REDIS_HOST
```

Consume it as env vars

```
# A · one key -> one var (explicit)
env:
  - name: REDIS_HOST
    valueFrom:
      configMapKeyRef:
        name: python-app-config
        key: REDIS_HOST
```

```
# B · every key -> a var, same name
envFrom:
  - configMapRef:
      name: python-app-config
```

Same 200 response as ex03 — the value is now reusable, reviewable, and separate from the image.

The gotcha: env vars don't hot-reload

Editing a ConfigMap does NOT restart pods. Env vars are read once, at container start.

You must roll the Deployment yourself:

```
$ kubectl patch configmap python-app-config --type merge \  
    -p '{"data":{"REDIS_HOST":"redis-primary","LOG_LEVEL":"debug"}}'  
$ kubectl rollout restart deploy/python-app  
$ kubectl rollout status  deploy/python-app  
  
$ kubectl exec deploy/python-app -- sh -c 'echo $REDIS_HOST $LOG_LEVEL '  
+redis-primary  debug
```

- Exception: a ConfigMap mounted as files DOES update in place (after a short kubelet sync) — the app just has to re-read the file.

Credentials: Docker Hub pull secret

```
type: kubernetes.io/dockerconfigjson
$ kubectl create secret docker-registry dockerhub-creds \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=<user> \
  --docker-password=<access-token>

spec:                                # in the pod / deployment
  imagePullSecrets:
    - name: dockerhub-creds
```

- No secret, private image →**
ErrImagePull / ImagePullBackOff
"pull access denied ..."
- base64 is encoding, not encryption
 - RBAC-restrict get/list on secrets
 - bind to default ServiceAccount → cluster-wide

	ConfigMap	Secret
for	plain, non-sensitive config	credentials, tokens, TLS keys
stored as	plain text	base64 — recoverable, NOT encrypted at rest by default
kinds	just one	Opaque · dockerconfigjson · tls · basic-auth ...

RECAP — THE MENTAL MODEL

Deployment runs it · Service finds it · ConfigMap/Secret configure it

- ex01** Deployment keeps N pods; Service + selector expose them
- ex02** Pods reach each other by Service DNS name
- ex03** env: makes the app's config explicit and testable
- ex04** ConfigMap externalises it — remember rollout restart
- ex05** Secret holds credentials; imagePullSecrets for registries

pocket cheat sheet

```
kubectl get all
kubectl describe pod <p>      # events!
kubectl logs -f deploy/<d>
kubectl exec -it deploy/<d> -- sh
kubectl rollout restart deploy/<d>
kubectl set env deploy/<d> K=v
kubectl get events --sort-by=.lastTimestamp
```

Next: probes & resources, Ingress (Traefik), PersistentVolumes, and Helm.